

LECTURE 24

Database Authorization

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

What We'll Cover

- Why database security matters: confidentiality, integrity, availability
- Database users and privileges
- Discretionary vs. mandatory access control, briefly
- Granting privileges with GRANT ... ON ... TO ...
- Revoking privileges with REVOKE
- Roles and role-based access control
- Views as a security mechanism, tying back to Lecture 23
- SQL injection and the least-privilege principle

Database Security and Authorization

Authorization decides who may do what to which data. Authentication proves who you are; authorization is enforced on every subsequent operation.



Confidentiality

Only authorized users can READ the data



Integrity

Only authorized users can MODIFY the data,
only in permitted ways



Availability

Authorized users can access data when they
need it

The CIA Triad

Database Users and Privileges

SELECT

Read rows from a table or view

INSERT

Add new rows

UPDATE

Modify existing rows

DELETE

Remove rows

EXECUTE

Run a stored procedure or function

ALL PRIVILEGES

Shorthand for every applicable privilege

Discretionary vs. Mandatory Access Control

Discretionary (DAC)

- The owner of an object (or an admin) decides, at their discretion, who gets which privileges
- This is what GRANT and REVOKE implement
- What almost every relational DBMS uses by default

Mandatory (MAC)

- Every object and user carries a fixed security classification (unclassified, secret, top secret)
- Access decided by comparing classifications per a system-wide policy — individual users cannot override it
- Appears mainly in government/military-grade systems; this course focuses on DAC

Granting Privileges

GRANT <privileges> ON <object> TO <user | role>

```
GRANT SELECT ON Staff TO usman;
```

```
GRANT SELECT, INSERT, UPDATE ON Staff  
  TO usman, ayesha;
```

```
GRANT ALL PRIVILEGES ON Branch TO db_admin;
```

- Gives one or more privileges, on one object, to one or more users or roles
- GRANT gives the recipient use of the privilege — not the ability to pass it along, unless WITH GRANT OPTION is added

Column-Level Grants and GRANT OPTION

Column-level: restrict to specific columns

```
GRANT UPDATE (position, salary) ON Staff  
  TO hr_clerk;
```

EXECUTE applies to procedures/functions

```
GRANT EXECUTE ON ProcessPayroll  
  TO payroll_service;
```

WITH GRANT OPTION lets the recipient re-grant

```
GRANT SELECT ON Staff TO branch_manager  
  WITH GRANT OPTION;
```

GRANT OPTION spreads responsibility, not just access — privilege chains can grow past what a single audit of top-level GRANTS makes obvious. Use it sparingly.

Revoking Privileges

REVOKE <privileges> ON <object> FROM <user | role>

```
REVOKE UPDATE ON Staff FROM usman;
```

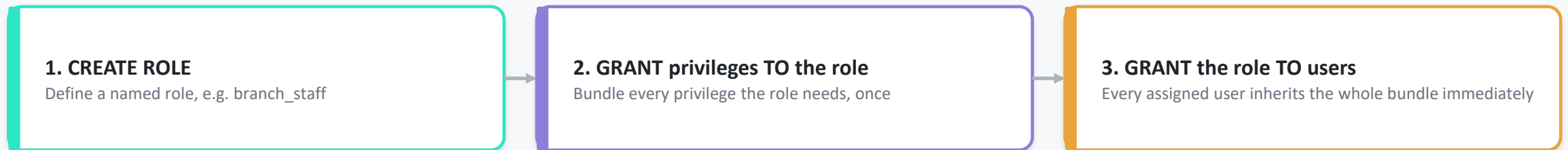
```
REVOKE ALL PRIVILEGES ON Branch FROM db_admin;
```

```
REVOKE SELECT ON Staff FROM branch_manager CASCADE;
```

- The mirror image of GRANT, with matching syntax
- Revoking a privilege granted WITH GRANT OPTION typically cascades — privileges granted onward because of it are revoked too
- RESTRICT can block a revoke that would cascade, where the system supports it

Roles and Role-Based Access Control

Granting to dozens of individual users doesn't scale — RBAC inserts one layer: define a role as a named bundle of privileges, assign users to roles.



```
CREATE ROLE branch_staff;
GRANT SELECT, INSERT, UPDATE ON Staff TO branch_staff;
GRANT SELECT ON Branch TO branch_staff;
GRANT branch_staff TO usman, ayesha, hamza;

GRANT DELETE ON Staff TO branch_staff; -- everyone gets it
REVOKE branch_staff FROM hamza;        -- loses it all, one statement
```

Authorization Rules

- An authorization rule is the DBMS's internal record of exactly which privilege a user (or role) holds on a specific object
- Effectively a row in the DBMS's own system catalog: (grantor, grantee, object, privilege, grantable?)
- Every SQL statement is checked against these rules BEFORE execution — no matching rule means rejection outright

Authorization is not a one-time login check — it is continuously enforced, on every single statement, every single time.

Views for Security: Restricting Columns

Hide salary from anyone who isn't HR

```
CREATE VIEW StaffPublicInfo AS
  SELECT staffNo, name, position, branchNo
  FROM Staff;

GRANT SELECT ON StaffPublicInfo TO general_staff;
-- general_staff never gets SELECT on Staff itself,
-- so salary stays invisible
```

- Privileges are granted on the VIEW, not the underlying table — the restriction is structural, not just application-level discipline
- StaffPublicInfo is single-table, no-aggregate — it remains updatable under Lecture 23's rules

Views for Security: Restricting Rows

Let each branch manager see only their own branch

```
CREATE VIEW MyBranchStaff AS
  SELECT staffNo, name, position, salary
  FROM Staff
  WHERE branchNo = CURRENT_BRANCH();
  -- resolved per session/user in a real system

GRANT SELECT ON MyBranchStaff TO branch_manager;
```

Because the branch manager is never granted any privilege on Staff directly, there is no way to bypass the WHERE filter by querying the base table instead — regardless of what tool or client they use.

SQL Authorization Commands — Consolidated

Command	Purpose
GRANT <priv> ON <obj> TO <user role>	Give one or more privileges on an object
GRANT ... WITH GRANT OPTION	Give privileges, and permission to re-grant them
REVOKE <priv> ON <obj> FROM <user role>	Remove previously granted privileges
REVOKE ... CASCADE	Remove privileges, and cascade to anything granted onward
CREATE ROLE <name>	Define a new named role
GRANT <role> TO <user>	Assign a user to a role, inheriting its whole bundle
REVOKE <role> FROM <user>	Remove a user from a role
CREATE VIEW ... AS SELECT ...	Define a restricted row/column subset to grant on

SQL Injection: the Most Common Real-World Attack

Untrusted input concatenated directly into SQL

```
-- Application builds this string directly from user input:
"SELECT * FROM Staff WHERE staffNo = '" + userInput + "'"

-- Attacker submits, as userInput:  ' OR '1'='1

-- Resulting statement actually executed:
SELECT * FROM Staff WHERE staffNo = ' OR '1'='1'
-- returns EVERY row, bypassing the intended filter
```

The fix: parameterized queries / prepared statements

```
PREPARE getStaff AS SELECT * FROM Staff WHERE staffNo = $1;
EXECUTE getStaff('SG37');
-- user input is bound as a parameter, never parsed as SQL
```

The Least-Privilege Principle

Grant a user (or an application's own database account) exactly the privileges its job requires — and nothing more.

- A reporting dashboard's database account needs SELECT and nothing else — it should never hold DELETE on any table
- So a bug or a successful injection attack against that dashboard cannot destroy data it was never supposed to touch
- Combined with roles (grant the role least privilege, once) and security views (expose only what a role needs), least privilege turns authorization into a system-wide, continuously-enforced discipline

Key Takeaways

- Authorization continuously decides who may do what to which data, resting on the CIA triad — confidentiality, integrity, availability.
- SQL implements discretionary access control: GRANT assigns privileges, REVOKE removes them, WITH GRANT OPTION lets a recipient re-grant.
- Roles bundle privileges once and let you assign or revoke an entire policy for a user in a single statement — grant to roles, assign users to roles.
- Views double as a genuine security mechanism: grant on a restricted view instead of the base table for row- and column-level restrictions, structurally enforced.
- SQL injection remains one of the most common real-world attacks; parameterized queries prevent it, and least privilege limits the damage even when some other defense fails.